

Persamaan yang disimulasikan adalah persamaan yang diambil dari artikel yang berjudul “**Macrophage response to Mycobacterium tuberculosis infection**” oleh D. Gammack · C.R. Doering · D.E. Kirschner tahun 2004.

Persamaan yang disimulasikan sebagai berikut :

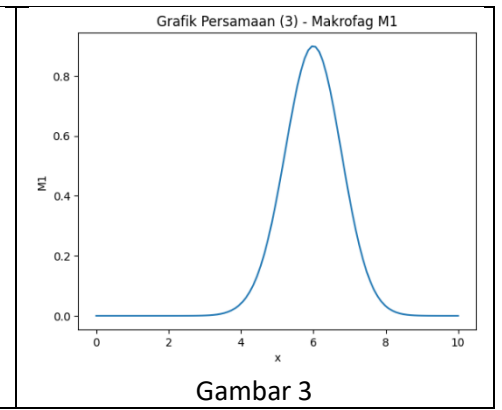
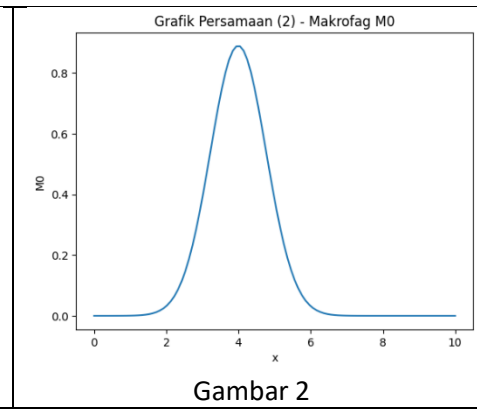
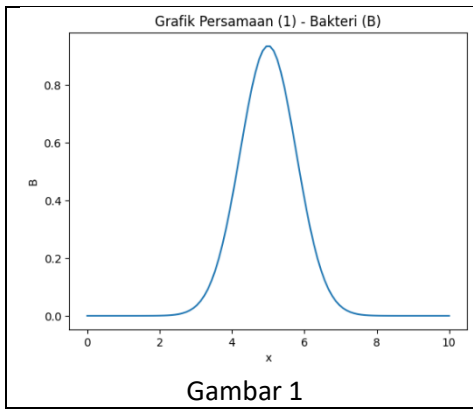
Persamaan 1, 2 dan 3:

$$\left. \begin{aligned} \frac{\partial B}{\partial t} + \underbrace{\frac{\partial (v B)}{\partial x}}_{\text{internal velocity}} &= \underbrace{\alpha B}_{\text{bacterial growth}} - \underbrace{\lambda B \sum_{w=0}^p M_w}_{\text{phagocytosis}} + \underbrace{\mu \sum_{w=0}^N w M_w}_{\text{macrophage death}} \\ &+ \underbrace{\beta_N N M_N}_{\text{macrophage bursting}} + \underbrace{D_B \frac{\partial^2 B}{\partial x^2}}_{\text{diffusion}} \end{aligned} \right\} \quad (1)$$

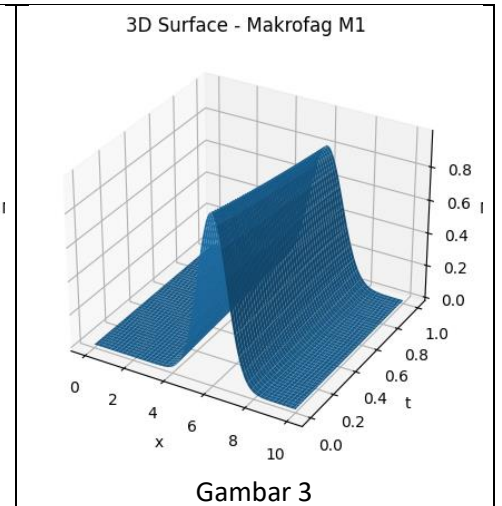
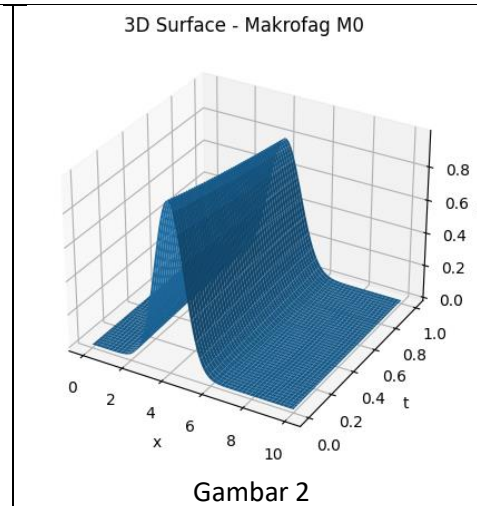
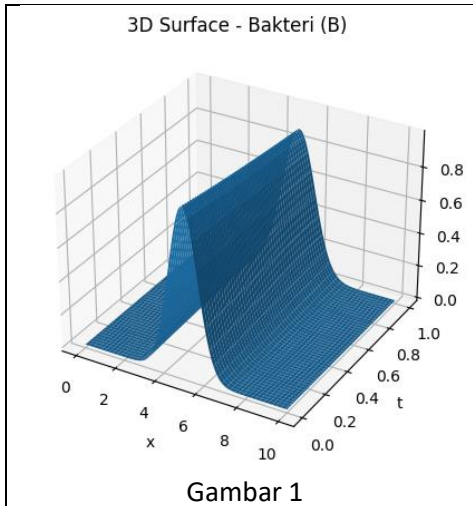
$$\left. \begin{aligned} \frac{\partial M_0}{\partial t} + \underbrace{\frac{\partial (v M_0)}{\partial x}}_{\text{internal velocity}} &= - \underbrace{\lambda B M_0}_{\text{phagocytosis}} - \underbrace{\mu M_0}_{\text{macrophage death}} + \underbrace{\psi \sum_{w=1}^{\kappa} M_w}_{\text{intracellular bacterial killing}} \\ &+ \underbrace{D_M \frac{\partial^2 M_0}{\partial x^2}}_{\text{diffusion}} - \underbrace{\frac{\partial}{\partial x} \chi \left(M_0 \frac{\partial C}{\partial x} \right)}_{\text{chemotaxis}} \end{aligned} \right\} \quad (2)$$

$$\left. \begin{aligned} \frac{\partial M_1}{\partial t} + \frac{\partial (v M_1)}{\partial x} &= \underbrace{-\lambda B M_1 + \lambda B M_0}_{\text{loss/gain due to phagocytosis}} - \mu M_1 - \psi M_1 \\ &- \underbrace{\beta_1 M_1}_{\text{intracellular bacteria growth}} + D_M \frac{\partial^2 M_1}{\partial x^2} \\ \frac{\partial M_w}{\partial t} + \frac{\partial (v M_w)}{\partial x} &= -\lambda_1 B M_w + \lambda_2 B M_{w-1} - \mu M_w - \psi_1 M_w \\ &- \underbrace{\beta_w w M_w + \beta_{w-1} (w-1) M_{w-1}}_{\text{loss/gain due to intracellular bacteria growth}} + D_M \frac{\partial (M_w)}{\partial x^2} \end{aligned} \right\} \quad (3)$$

dari persamaan 1, 2 dan 3 dihasilkan grafik seperti gambar 1, gambar 2 dan gambar 3



gambar 3D



Berikut script python:

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

# =====
# 1. GRID SPASIAL & WAKTU
# =====
x = np.linspace(0, 10, 100) # ruang
t = np.linspace(0, 2, 80)    # waktu

dx = x[1] - x[0]
dt = t[1] - t[0]

# =====
# 2. KONDISI AWAL
# =====
B = np.exp(-((x - 5)**2))    # bakteri
M0 = np.exp(-((x - 4)**2))   # makrofag awal
M1 = np.exp(-((x - 6)**2))   # makrofag aktif

# =====
# 3. PARAMETER MODEL
# =====
alpha = 0.2 # pertumbuhan bakteri
lam = 0.1 # laju fagositosis
mu = 0.05 # kematian makrofag
```

```

DB = 0.2 # difusi bakteri
DM = 0.15 # difusi makrofag

# =====
# 4. OPERATOR LAPLACIAN
# =====
def laplacian(u):
    return (np.roll(u, -1) - 2*u + np.roll(u, 1)) / dx**2

# =====
# 5. SIMULASI NUMERIK
# =====
B_hist = []
M0_hist = []
M1_hist = []

for _ in t:
    # Simpan kondisi
    B_hist.append(B.copy())
    M0_hist.append(M0.copy())
    M1_hist.append(M1.copy())

    # Update persamaan
    B_new = B + dt * (alpha*B - lam*B + DB*laplacian(B))
    M0_new = M0 + dt * (-lam*B*M0 - mu*M0 + DM*laplacian(M0))
    M1_new = M1 + dt * (lam*B*M0 - mu*M1 + DM*laplacian(M1))

    B, M0, M1 = B_new, M0_new, M1_new

# Convert ke array
B_hist = np.array(B_hist)
M0_hist = np.array(M0_hist)
M1_hist = np.array(M1_hist)

# Meshgrid untuk plotting
X, T = np.meshgrid(x, t)

# =====
# 6. PLOT 3D SURFACE
# =====
def plot_surface(Z, title, xlabel):
    fig = plt.figure()
    ax = fig.add_subplot(111, projection='3d')

    ax.plot_surface(X, T, Z)

    ax.set_title(title)
    ax.set_xlabel("Ruang (x)")
    ax.set_ylabel("Waktu (t)")
    ax.set_zlabel(xlabel)

    plt.show()

# =====
# 7. TAMPILKAN GRAFIK
# =====
plot_surface(B_hist, "3D Surface Bakteri (B)", "B")
plot_surface(M0_hist, "3D Surface Makrofag M0", "M0")

```

```
plot_surface(M1_hist, "3D Surface Makrofag M1", "M1")
```

Persamaan 4:

similarly with intracellular bacteria growth. Infected macrophages, with a bacterial load $w \leq \kappa$, clear this load at a rate ψ and return to the uninfected macrophage state, M_0 . In the infected macrophage equation this is represented by the following form of $\psi_1(w)$

$$\psi_1(w) = \begin{cases} \psi, & w \leq \kappa \\ 0, & \text{otherwise} \end{cases}$$

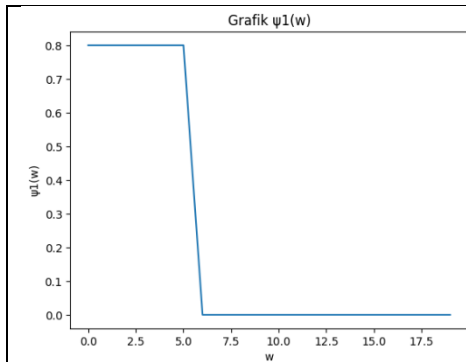
Macrophages with a bacterial load $w \leq p$ can perform phagocytosis, and that rate is represented by $\lambda_1(w)$ and $\lambda_2(w)$ as follows

$$\lambda_1(w) = \begin{cases} \lambda, & w \leq p \\ 0, & \text{otherwise} \end{cases}, \quad \lambda_2(w) = \begin{cases} \lambda, & w \leq p + 1 \\ 0, & \text{otherwise} \end{cases}$$

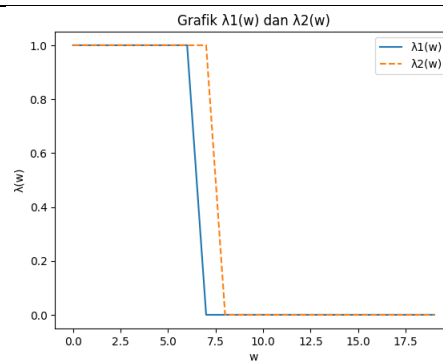
Finally, bacteria within macrophages grow at a rate that decreases with the number of bacteria within the host due to space and nutritional constraints. Hence, we define $\beta_w(w)$ as

$$\beta_w(w) = \frac{\beta_0}{(w_0 + w)}. \quad (4)$$

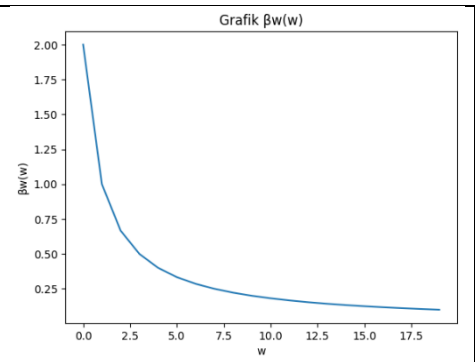
Hasil dari persamaan 4



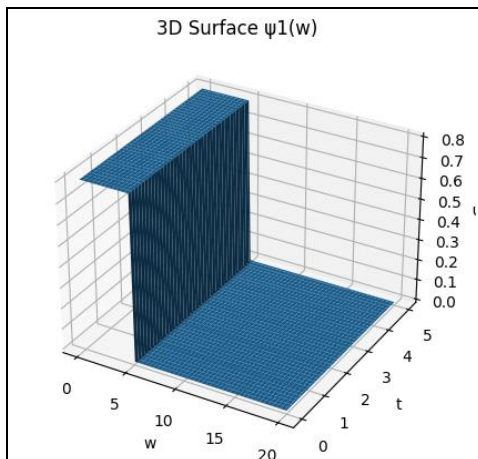
Gambar 4



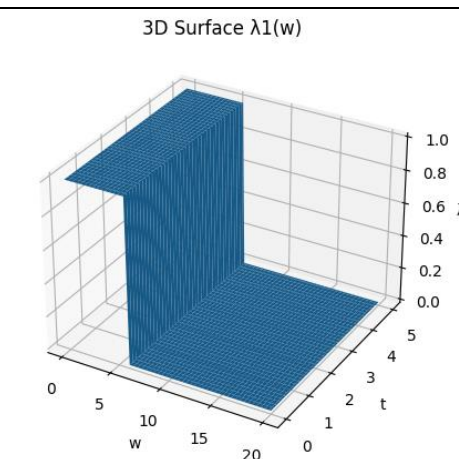
Gambar 5



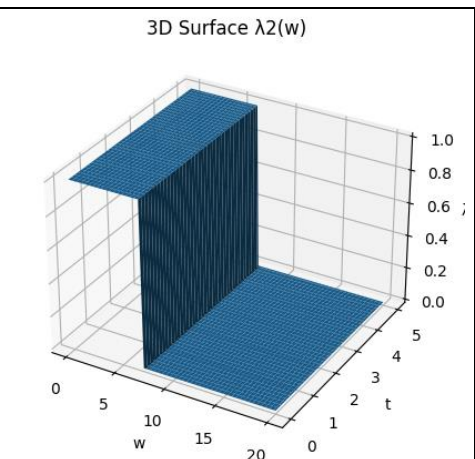
Gambar 6



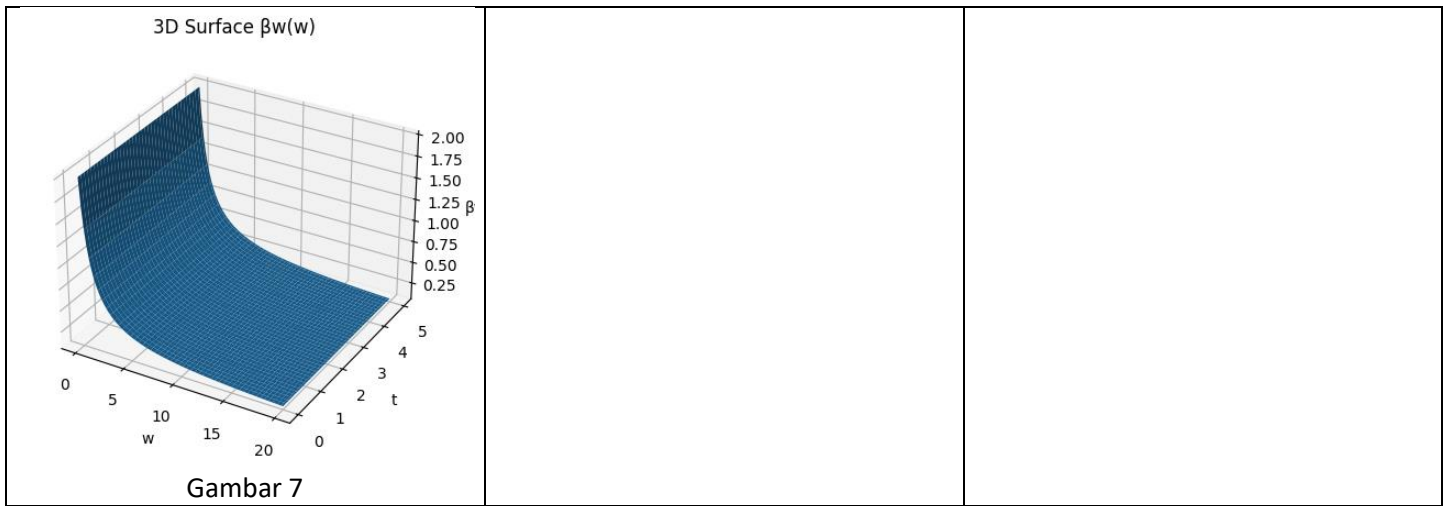
Gambar 4



Gambar 5



Gambar 6



```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

# =====
# 1. DOMAIN
# =====
w = np.linspace(0, 20, 100) # jumlah bakteri
t = np.linspace(0, 5, 80)   # waktu

W, T = np.meshgrid(w, t)

# =====
# 2. PARAMETER MODEL
# =====
k = 5
p = 6
lam = 1.0
psi = 0.8
beta0 = 2.0
w0 = 1.0

# =====
# 3. DEFINISI FUNGSI
# =====

# fungsi killing
psi1 = np.where(W <= k, psi, 0)

# fungsi fagositosis
lambda1 = np.where(W <= p, lam, 0)
lambda2 = np.where(W <= p + 1, lam, 0)

# fungsi pertumbuhan
beta_w = beta0 / (w0 + W)

# =====
# 4. FUNGSI PLOT 3D
# =====
def plot_3d(Z, title, zlabel):
    fig = plt.figure()
    ax = fig.add_subplot(111, projection='3d')
```

```

ax.plot_surface(W, T, Z)

ax.set_title(title)
ax.set_xlabel("w (jumlah bakteri)")
ax.set_ylabel("t (waktu)")
ax.set_zlabel(zlabel)

plt.show()

# =====
# 5. VISUALISASI
# =====
plot_3d(psi1, "3D  $\psi_1(w)$  - Killing Function", " $\psi_1(w)$ ")
plot_3d(lambda1, "3D  $\lambda_1(w)$  - Phagocytosis", " $\lambda_1(w)$ ")
plot_3d(lambda2, "3D  $\lambda_2(w)$  - Phagocytosis", " $\lambda_2(w)$ ")
plot_3d(beta_w, "3D  $\beta w(w)$  - Growth Function", " $\beta w(w)$ ")

```

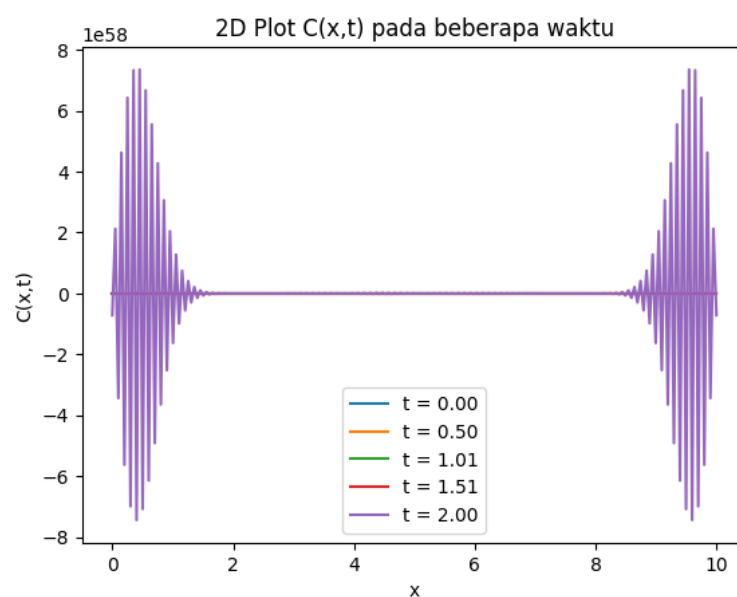
Persamaan 5:

Chemoattractant dynamics

The chemoattractant $C(x, t)$ is produced by extracellular bacteria, B , at a maximal rate s_B and diffuses at a rate D_C . It has a natural decay rate, Γ_C , and is utilized by uninfected macrophages, M_0 , at a rate Γ_M . The governing equation for chemoattractant, C is:

$$\frac{\partial C}{\partial t} = s_B \frac{B}{B + b_0} - \Gamma_C C - \Gamma_M M_0 C + D_C \frac{\partial^2 C}{\partial x^2}. \quad (5)$$

Persamaan 5 2D:



Gambar 8

```

import numpy as np
import matplotlib.pyplot as plt

# Grid (dibuat lebih stabil)

```

```

x = np.linspace(0, 10, 200)
t = np.linspace(0, 1, 200)

dx = x[1] - x[0]
dt = t[1] - t[0]

# Initial condition
C = np.zeros_like(x)
B = np.exp(-((x-5)**2))
M0 = np.exp(-((x-4)**2))

# Parameter
sB = 1.0
b0 = 1.0
GammaC = 0.3
GammaM = 0.2
DC = 0.1 # diturunkan untuk stabilitas

def laplacian(u):
    return (np.roll(u, -1) - 2*u + np.roll(u, 1)) / dx**2

# Snapshot waktu
snapshots = []
times_to_plot = [0.1, 0.3, 0.5, 0.8, 1.0]

for ti in t:
    if any(abs(ti - tp) < dt for tp in times_to_plot):
        snapshots.append((ti, C.copy()))

    production = sB * (B / (B + b0))
    decay = -GammaC * C
    uptake = -GammaM * M0 * C
    diffusion = DC * laplacian(C)

    C = C + dt * (production + decay + uptake + diffusion)

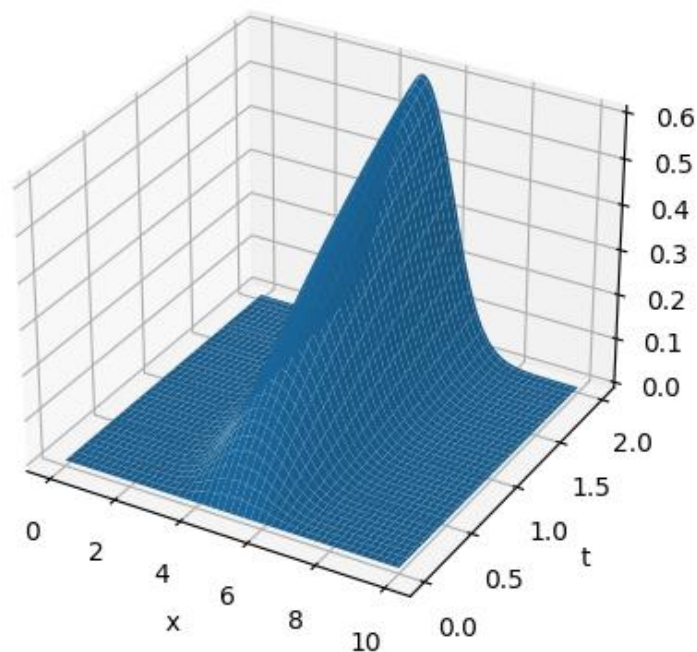
# Plot 2D
plt.figure()
for ti, snap in snapshots:
    plt.plot(x, snap, label=f"t = {ti:.2f}")

plt.title("C(x,t) Stabil (2D)")
plt.xlabel("x")
plt.ylabel("C")
plt.legend()
plt.show()

```

Persamaan 3 3D:

3D Surface Chemotactant $C(x,t)$



Gambar 9

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

# =====
# 1. GRID (STABIL)
# =====
x = np.linspace(0, 10, 150)
t = np.linspace(0, 1, 150)

dx = x[1] - x[0]
dt = t[1] - t[0]

# =====
# 2. KONDISI AWAL
# =====
C = np.zeros_like(x)

# Distribusi bakteri & makrofag
B = np.exp(-((x - 5)**2))
M0 = np.exp(-((x - 4)**2))

# =====
# 3. PARAMETER MODEL
# =====
sB = 1.0
b0 = 1.0
GammaC = 0.3
GammaM = 0.2
DC = 0.1 # penting untuk stabilitas

# =====
```



```

# 4. LAPLACIAN (DIFUSI)
# =====
def laplacian(u):
    return (np.roll(u, -1) - 2*u + np.roll(u, 1)) / dx**2

# =====
# 5. SIMULASI
# =====
C_hist = []

for _ in t:
    C_hist.append(C.copy())

    production = sB * (B / (B + b0))
    decay = -GammaC * C
    uptake = -GammaM * M0 * C
    diffusion = DC * laplacian(C)

    C = C + dt * (production + decay + uptake + diffusion)

C_hist = np.array(C_hist)

# =====
# 6. VISUALISASI 3D
# =====
X, T = np.meshgrid(x, t)

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

ax.plot_surface(X, T, C_hist)

ax.set_title("3D Surface Chemotactant C(x,t)")
ax.set_xlabel("x (ruang)")
ax.set_ylabel("t (waktu)")
ax.set_zlabel("C(x,t)")

plt.show()

```

Persamaan 6 dan 7:

Granuloma radius

We assume that cellular velocity, $v(x, t)$ at the granuloma boundary, $x = R(t)$, describes the radial growth of the granuloma and thus the equation governing R is

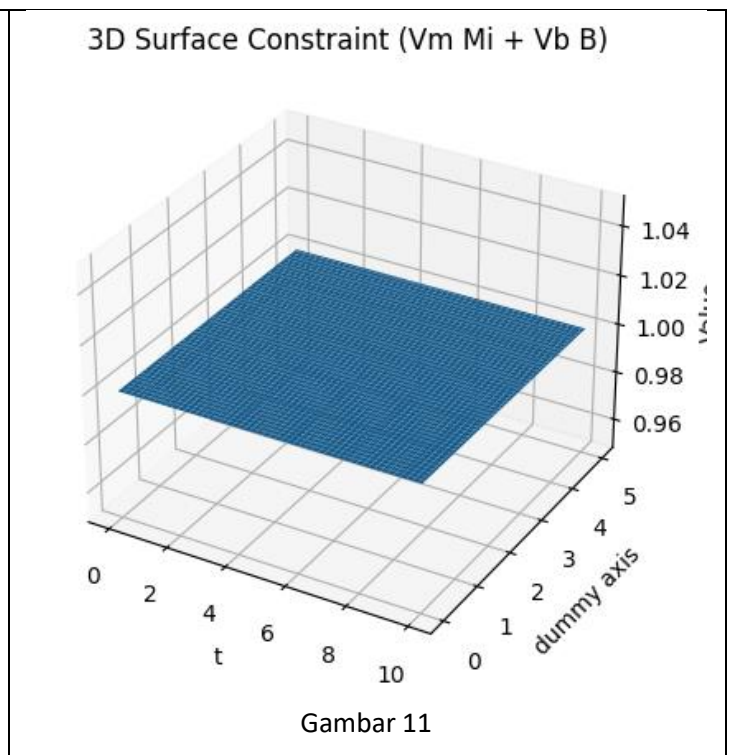
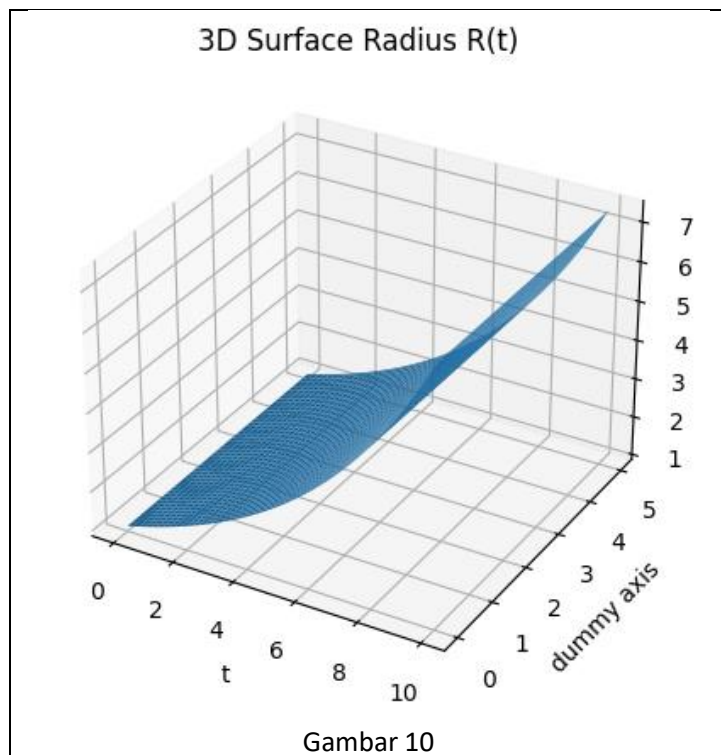
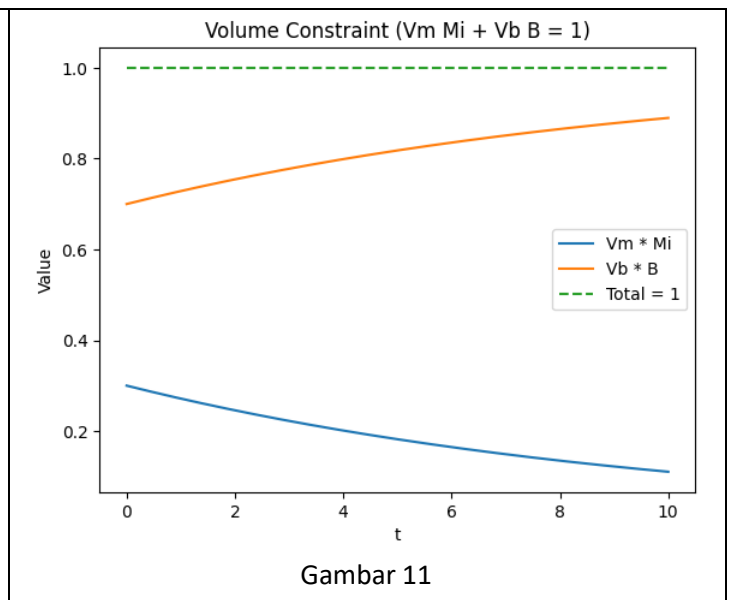
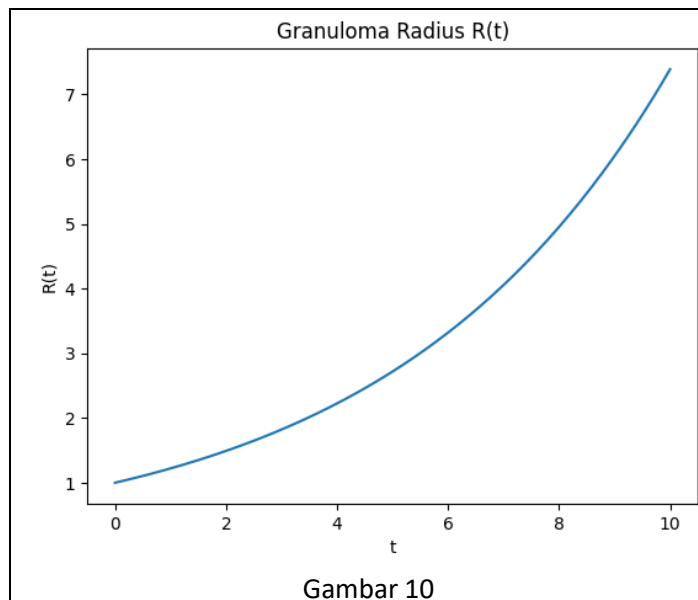
$$\frac{\partial R}{\partial t} = v(R, t). \quad (6)$$

Finally, we assume that there are no voids in the granuloma, therefore, for $x \leq R(t)$

$$V_M \sum_{i=0}^N M_i + V_B B = 1, \quad (7)$$

where V_M and V_B are, respectively, the volume of a single macrophage and a single bacterium.

Hasil dari persamaan



```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

# =====
# 1. DOMAIN
# =====
t = np.linspace(0, 10, 150) # waktu
y = np.linspace(0, 5, 100) # dummy axis (untuk surface)

T, Y = np.meshgrid(t, y)

# =====
# 2. MODEL RADIUS R(t)
# =====
```

```
# Model realistis (logistic growth, lebih baik dari eksponensial)
```

```
R_max = 10
```

```
k = 0.5
```

```
R0 = 1
```

```
R = R_max / (1 + ((R_max - R0)/R0) * np.exp(-k * T))
```

```
# =====
```

```
# 3. MODEL POPULASI
```

```
# =====
```

```
# Makrofag menurun
```

```
M = 0.6 * np.exp(-0.2 * T)
```

```
# Volume parameter
```

```
Vm = 0.6
```

```
Vb = 0.4
```

```
# Bakteri dari constraint
```

```
B = (1 - Vm * M) / Vb
```

```
# Constraint total
```

```
constraint = Vm * M + Vb * B
```

```
# =====
```

```
# 4. PLOT 3D R(t)
```

```
# =====
```

```
fig = plt.figure()
```

```
ax = fig.add_subplot(111, projection='3d')
```

```
ax.plot_surface(T, Y, R)
```

```
ax.set_title("3D Radius Granuloma R(t)")
```

```
ax.set_xlabel("Waktu (t)")
```

```
ax.set_ylabel("Dimensi tambahan")
```

```
ax.set_zlabel("Radius R")
```

```
plt.show()
```

```
# =====
```

```
# 5. PLOT 3D CONSTRAINT
```

```
# =====
```

```
fig = plt.figure()
```

```
ax = fig.add_subplot(111, projection='3d')
```

```
ax.plot_surface(T, Y, constraint)
```

```
ax.set_title("3D Constraint Volume (Vm M + Vb B = 1)")
```

```
ax.set_xlabel("Waktu (t)")
```

```
ax.set_ylabel("Dimensi tambahan")
```

```
ax.set_zlabel("Nilai")
```

```
plt.show()
```

Persamaan 8:

For ease of numerical analysis we non-dimensionalize equations (1)–(7). The bacteria and macrophage densities are non-dimensionalized with respect to the volume of a single bacterium (V_B) and macrophage (V_M), respectively. Time is non-dimensionalized by the bacterial growth rate (α) while the spatial coordinate, x , and the granuloma radius, $R(t)$, are non-dimensionalized with respect to the initial granuloma radius $R(t_0) = R_0$. Finally, the chemoattractant is non-dimensionalized by s_B / Γ_C . Thus, the non-dimensionalizations are written as:

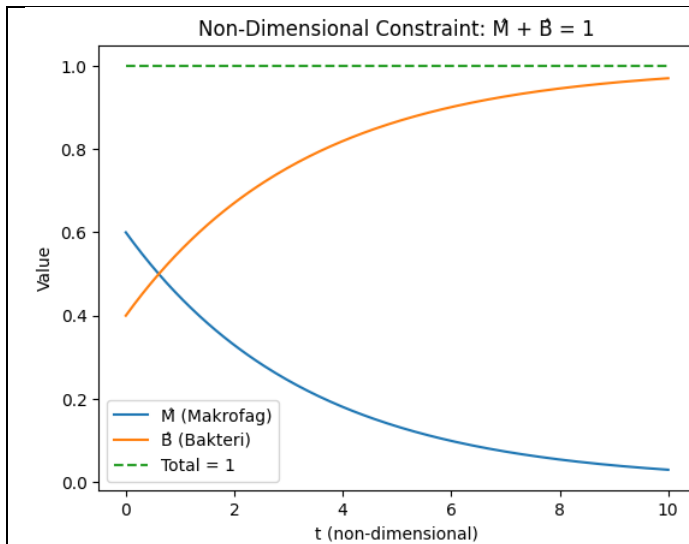
$$B = \hat{B} / V_B, \quad M_j = \hat{M}_j / V_M, \quad t = \hat{t} / \alpha, \quad x = \hat{x} R_0,$$

$$R = \hat{R} R_0, \quad v = \hat{v} R_0 \alpha, \quad C = \hat{C} (s_B / \Gamma_C)$$

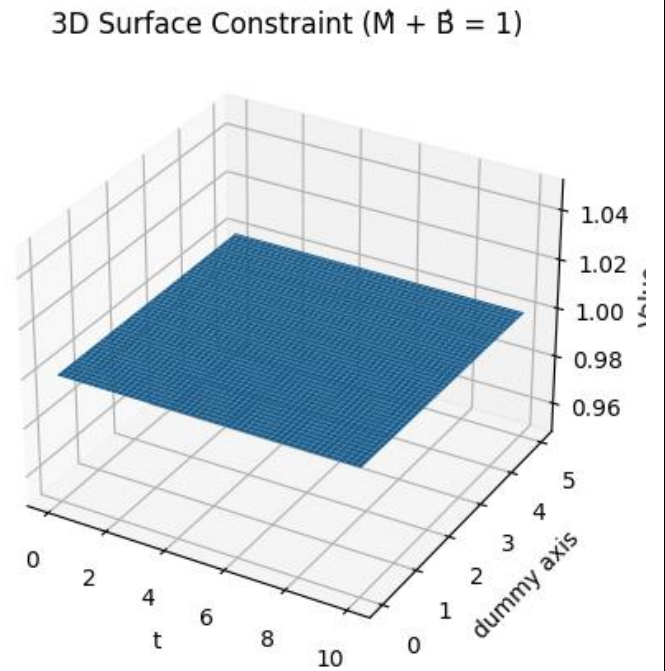
Now, using the non-dimensional no-voids condition (7), B is given by

$$\sum_{i=0}^N \hat{M}_i + \hat{B} = 1. \quad (8)$$

Hasil dari persamaan



Gambar 12



Gambar 12

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

# =====
# 1. DOMAIN WAKTU
# =====
t = np.linspace(0, 10, 200)

# =====
# 2. MODEL NON-DIMENSI
# =====
# Makrofag (menurun)
M_hat = 0.6 * np.exp(-0.3 * t)

# Bakteri (dari constraint)
B_hat = 1 - M_hat
```

```

# =====
# 3. GRAFIK 2D
# =====
plt.figure()

plt.plot(t, M_hat, label="M̂ (Makrofag)")
plt.plot(t, B_hat, label="B̂ (Bakteri)")
plt.plot(t, M_hat + B_hat, '--', label="Total = 1")

plt.title("Non-Dimensional Constraint: M̂ + B̂ = 1")
plt.xlabel("t (non-dimensional)")
plt.ylabel("Value")
plt.legend()

plt.show()

# =====
# 4. GRAFIK 3D
# =====
y = np.linspace(0, 5, 100) # dummy axis
T, Y = np.meshgrid(t, y)

# Extend ke 3D
M_surface = 0.6 * np.exp(-0.3 * T)
B_surface = 1 - M_surface
Total_surface = M_surface + B_surface

# Plot 3D
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

ax.plot_surface(T, Y, Total_surface)

ax.set_title("3D Surface Constraint (M̂ + B̂ = 1)")
ax.set_xlabel("t")
ax.set_ylabel("Dimensi tambahan")
ax.set_zlabel("Value")

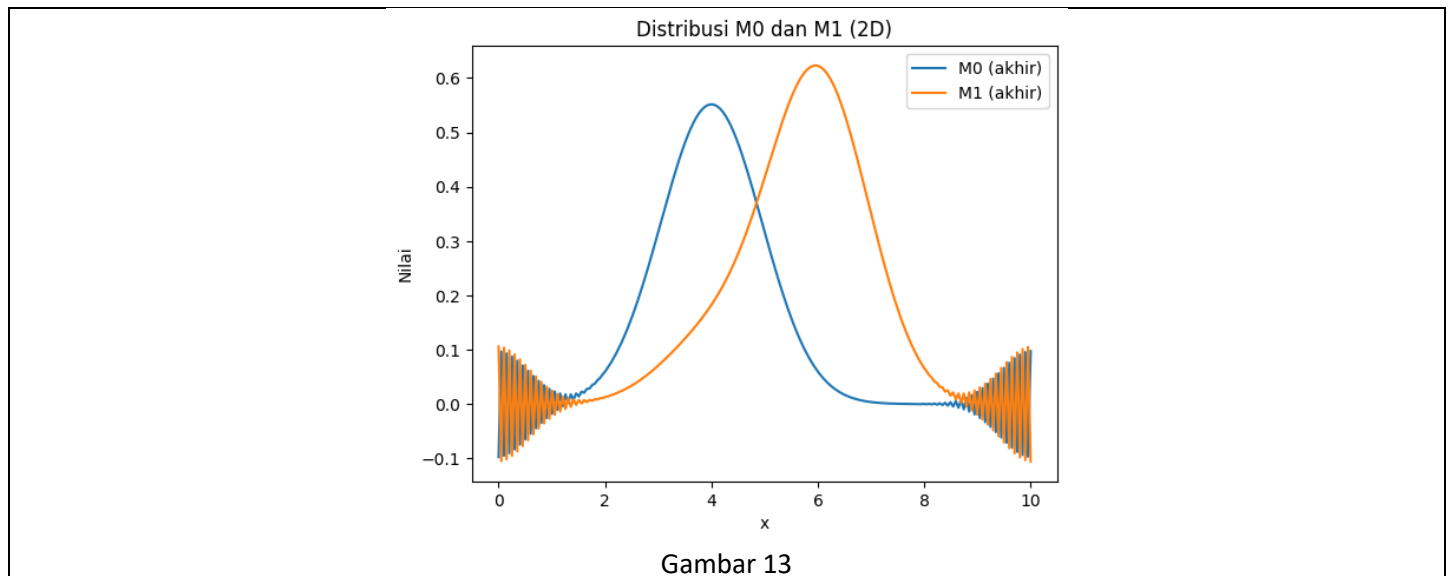
plt.show()

```

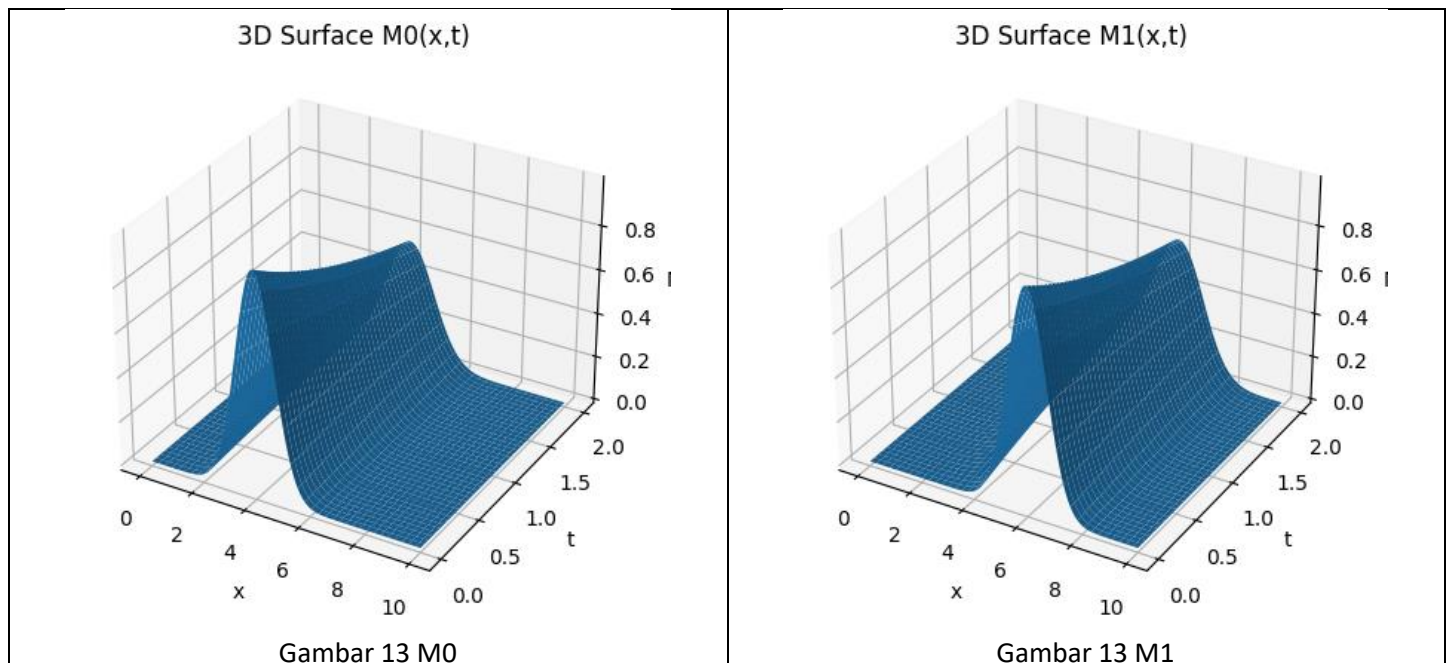
Persamaan 9:

$$\left. \begin{aligned}
 \frac{\partial \hat{M}_0}{\partial \hat{t}} + \frac{\partial (\hat{v} \hat{M}_0)}{\partial \hat{x}} &= -\hat{\lambda} \hat{B} \hat{M}_0 - \hat{\mu} \hat{M}_0 + \hat{\psi} \sum_{j=1}^{\kappa} \hat{M}_j \\
 &\quad + \hat{D}_M \frac{\partial^2 \hat{M}_0}{\partial \hat{x}^2} - \hat{\chi} \frac{\partial}{\partial \hat{x}} \left(\hat{M}_0 \frac{\partial \hat{C}}{\partial \hat{x}} \right) \\
 \frac{\partial \hat{M}_1}{\partial \hat{t}} + \frac{\partial (\hat{v} \hat{M}_1)}{\partial \hat{x}} &= -\hat{\lambda} \hat{B} \hat{M}_1 + \hat{\lambda} \hat{B} \hat{M}_0 - \hat{\mu} \hat{M}_1 - \hat{\psi} \hat{M}_1 \\
 &\quad - \hat{\beta}_1 \hat{M}_1 + \hat{D}_M \frac{\partial^2 \hat{M}_1}{\partial \hat{x}^2} \\
 \frac{\partial \hat{M}_w}{\partial \hat{t}} + \frac{\partial (\hat{v} \hat{M}_w)}{\partial \hat{x}} &= -\hat{\lambda}_1 \hat{B} \hat{M}_w + \hat{\lambda}_2 \hat{B} \hat{M}_{w-1} - \hat{\mu} \hat{M}_w - \hat{\psi}_1 \hat{M}_w \\
 &\quad - \hat{\beta}_w w \hat{M}_w + \hat{\beta}_{w-1} (w-1) \hat{M}_{w-1} \\
 &\quad + \hat{D}_M \frac{\partial^2 \hat{M}_w}{\partial \hat{x}^2}
 \end{aligned} \right\} \quad (9)$$

Hasil dari persamaan



Gambar 13



Gambar 13 M0

Gambar 13 M1

```

import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

# =====
# 1. GRID
# =====
L = 10
Nx = 120
Nt = 150

x = np.linspace(0, L, Nx)
t = np.linspace(0, 2, Nt)

dx = x[1] - x[0]
dt = t[1] - t[0]

X, T = np.meshgrid(x, t)

# =====
# 2. KONDISI AWAL
# =====
M0 = np.exp(-((x - 4)**2)) # makrofag sehat
M1 = np.exp(-((x - 6)**2)) # makrofag terinfeksi

# bakteri (diasumsikan tetap untuk sederhana)
B = np.exp(-((x - 5)**2))

# =====
# 3. PARAMETER MODEL
# =====
lam = 0.15 # infeksi
mu = 0.05 # kematian
beta1 = 0.05 # degradasi M1
DM = 0.1 # difusi

# =====
# 4. OPERATOR DIFUSI
# =====
def laplacian(u):
    return (np.roll(u, -1) - 2*u + np.roll(u, 1)) / dx**2

# =====
# 5. SIMULASI
# =====
M0_hist = []
M1_hist = []

for _ in t:
    M0_hist.append(M0.copy())
    M1_hist.append(M1.copy())

# ===== Persamaan M0 =====
dM0 = (
    - lam * B * M0
    - mu * M0

```

```

    + DM * laplacian(M0)
)

# ===== Persamaan M1 =====
dM1 = (
    + lam * B * M0
    - mu * M1
    - beta1 * M1
    + DM * laplacian(M1)
)

# update
M0 = M0 + dt * dM0
M1 = M1 + dt * dM1

# ubah ke array
M0_hist = np.array(M0_hist)
M1_hist = np.array(M1_hist)

# =====
# 6. PLOT 3D M0
# =====
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

ax.plot_surface(X, T, M0_hist)

ax.set_title("3D Surface M0(x,t)")
ax.set_xlabel("x")
ax.set_ylabel("t")
ax.set_zlabel("M0")

plt.show()

# =====
# 7. PLOT 3D M1
# =====
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

ax.plot_surface(X, T, M1_hist)

ax.set_title("3D Surface M1(x,t)")
ax.set_xlabel("x")
ax.set_ylabel("t")
ax.set_zlabel("M1")

plt.show()

```

Persamaan 10, 11 dan 12:

The non-dimensional chemoattractant equation is

$$\frac{\partial \hat{C}}{\partial \hat{t}} = \frac{\hat{B}}{\hat{B} + \hat{b}_0} - \hat{\Gamma}_C \hat{C} - \hat{\Gamma}_M \hat{M}_0 \hat{C} + \hat{D}_C \frac{\partial^2 \hat{C}}{\partial \hat{x}^2}, \quad (10)$$

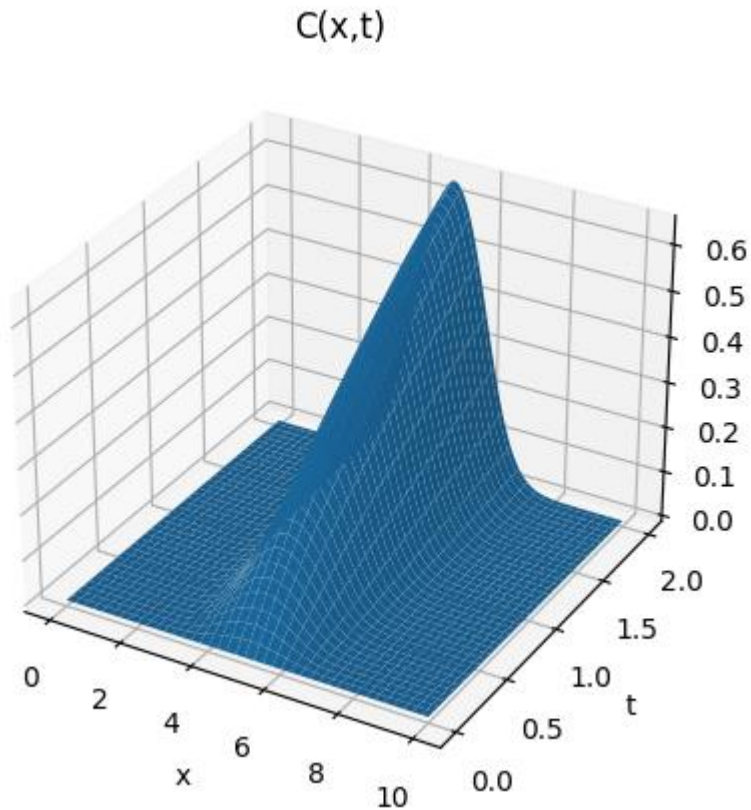
and the non-dimensional granuloma radius is given by

$$\frac{\partial \hat{R}}{\partial \hat{t}} = \hat{v}(\hat{R}, \hat{t}). \quad (11)$$

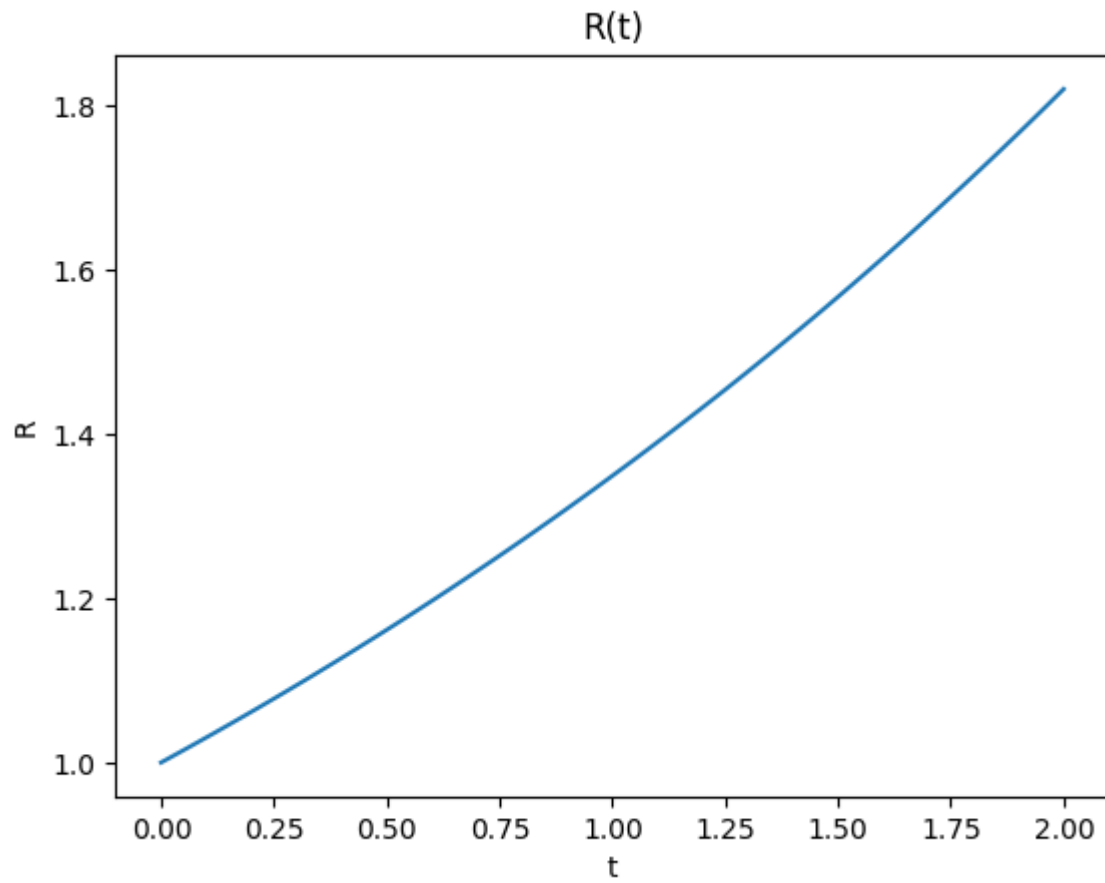
Finally, using the no-voids condition, equation (8), the equation governing the internal velocity can be written as

$$\left. \begin{aligned} \hat{v}_{\hat{x}} = & \hat{B} - V_r \hat{\lambda} \hat{B} \sum_{j=0}^N \hat{M}_j + \hat{\mu} \sum_{j=0}^N \left[\hat{M}_j (V_r j - 1) \right] \\ & + \hat{\beta}_N N (V_r - 1) \hat{M}_N + \hat{D}_B \frac{\partial^2 \hat{B}}{\partial \hat{x}^2} + \hat{D}_M \sum_{j=0}^N \frac{\partial^2 \hat{M}_j}{\partial \hat{x}^2} \\ & - \hat{\chi} \frac{\partial}{\partial \hat{x}} \left(\hat{M}_0 \frac{\partial \hat{C}}{\partial \hat{x}} \right) \end{aligned} \right\}, \quad (12)$$

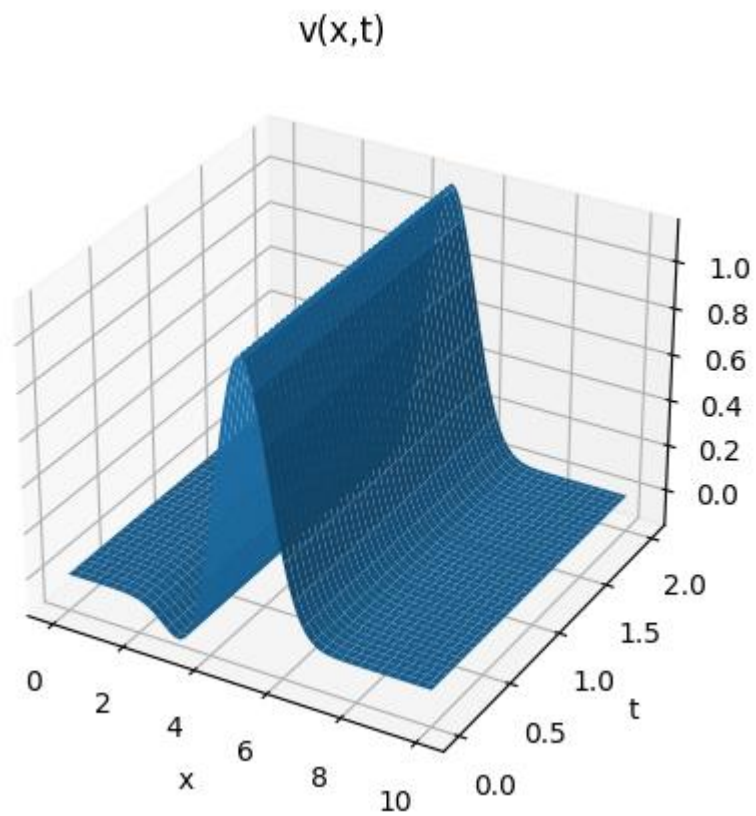
Hasil dari persamaan



Gambar 14



Gambar 15



Gambar 16

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

# =====
```

```

# 1. GRID
# =====
L = 10
Nx = 120
Nt = 150

x = np.linspace(0, L, Nx)
t = np.linspace(0, 2, Nt)

dx = x[1] - x[0]
dt = t[1] - t[0]

X, T = np.meshgrid(x, t)

# =====
# 2. KONDISI AWAL
# =====
C = np.zeros_like(x)
B = np.exp(-((x - 5)**2))
M0 = np.exp(-((x - 4)**2))

# =====
# 3. PARAMETER NON-DIMENSI
# =====
b0 = 1.0
GammaC = 0.3
GammaM = 0.2
DC = 0.1

# radius
R = 1.0
kR = 0.4

# velocity parameter
vr = 0.2
chi = 0.1
DB = 0.05
DM = 0.05

# =====
# 4. OPERATOR DIFUSI
# =====
def laplacian(u):
    return (np.roll(u, -1) - 2*u + np.roll(u, 1)) / dx**2

def gradient(u):
    return np.gradient(u, dx)

# =====
# 5. STORAGE
# =====
C_hist = []
R_hist = []
v_hist = []

# =====
# 6. SIMULASI
# =====

```

```

for _ in t:
    # simpan
    C_hist.append(C.copy())
    R_hist.append(R)

    # ===== (10) CHEMOATTRACTANT =====
    production = B / (B + b0)
    decay = -GammaC * C
    uptake = -GammaM * M0 * C
    diffusion = DC * laplacian(C)

    C = C + dt * (production + decay + uptake + diffusion)

    # ===== (11) RADIUS =====
    dR = kR * R
    R = R + dt * dR

    # ===== (12) VELOCITY (SIMPLIFIED) =====
    gradC = gradient(C)
    gradM0 = gradient(M0)

```

```

v = (
    B
    - vr * B * M0
    + DB * laplacian(B)
    + DM * laplacian(M0)
    - chi * gradient(M0 * gradC)
)

```

```

v_hist.append(v)

```

```

# ubah ke array
C_hist = np.array(C_hist)
R_hist = np.array(R_hist)
v_hist = np.array(v_hist)

```

```

# =====
# 7. PLOT 3D C(x,t)
# =====
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

```

```

ax.plot_surface(X, T, C_hist)

```

```

ax.set_title("3D Chemoattractant C(x,t)")
ax.set_xlabel("x")
ax.set_ylabel("t")
ax.set_zlabel("C")

```

```

plt.show()

```

```

# =====
# 8. PLOT R(t)
# =====
plt.figure()
plt.plot(t, R_hist)

```

```

plt.title("Granuloma Radius R(t)")

```

```
plt.xlabel("t")
plt.ylabel("R")

plt.show()

# =====
# 9. PLOT 3D VELOCITY v(x,t)
# =====
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

ax.plot_surface(X, T, v_hist)

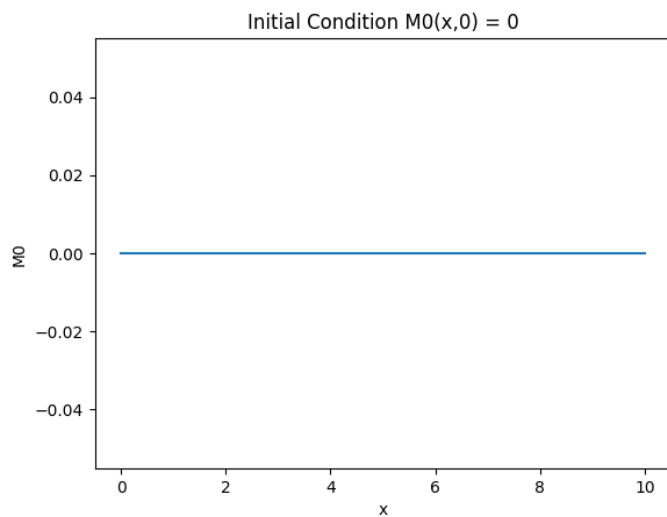
ax.set_title("3D Internal Velocity v(x,t)")
ax.set_xlabel("x")
ax.set_ylabel("t")
ax.set_zlabel("v")

plt.show()
```

Persamaan 13:

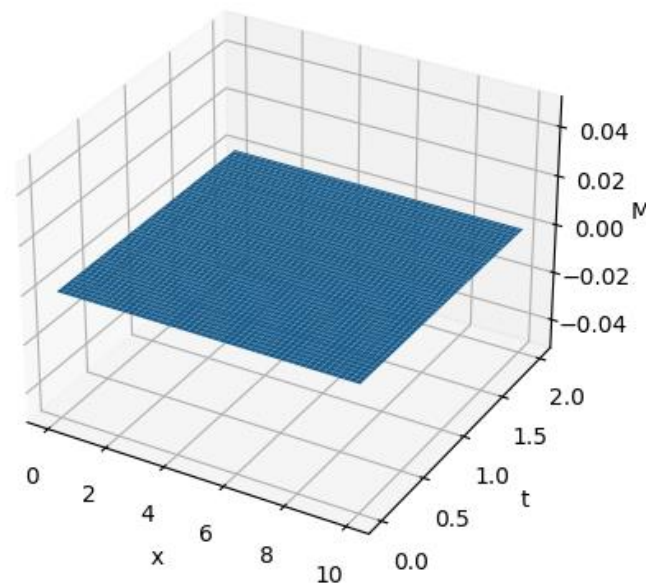
$$\hat{M}_0(x, 0) = 0.0. \quad (13)$$

Hasil dari persamaan



Gambar 17

3D Surface $M_0(x,t)$ with Initial Condition = 0



Gambar 17

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

# =====
# 1. DOMAIN
# =====
```

```

x = np.linspace(0, 10, 200) # ruang
t = np.linspace(0, 2, 100) # waktu

# =====
# 2. KONDISI AWAL
# =====
M0 = np.zeros_like(x) # M0(x,0) = 0

# =====
# 3. PLOT 2D
# =====
plt.figure()
plt.plot(x, M0)

plt.title("Kondisi Awal: M0(x,0) = 0")
plt.xlabel("x (ruang)")
plt.ylabel("M0")

plt.grid()
plt.show()

# =====
# 4. PLOT 3D (EXTENSION)
# =====
X, T = np.meshgrid(x, t)
M0_surface = np.zeros_like(X)

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

ax.plot_surface(X, T, M0_surface)

ax.set_title("3D Surface M0(x,t) dengan M0(x,0)=0")
ax.set_xlabel("x")
ax.set_ylabel("t")
ax.set_zlabel("M0")

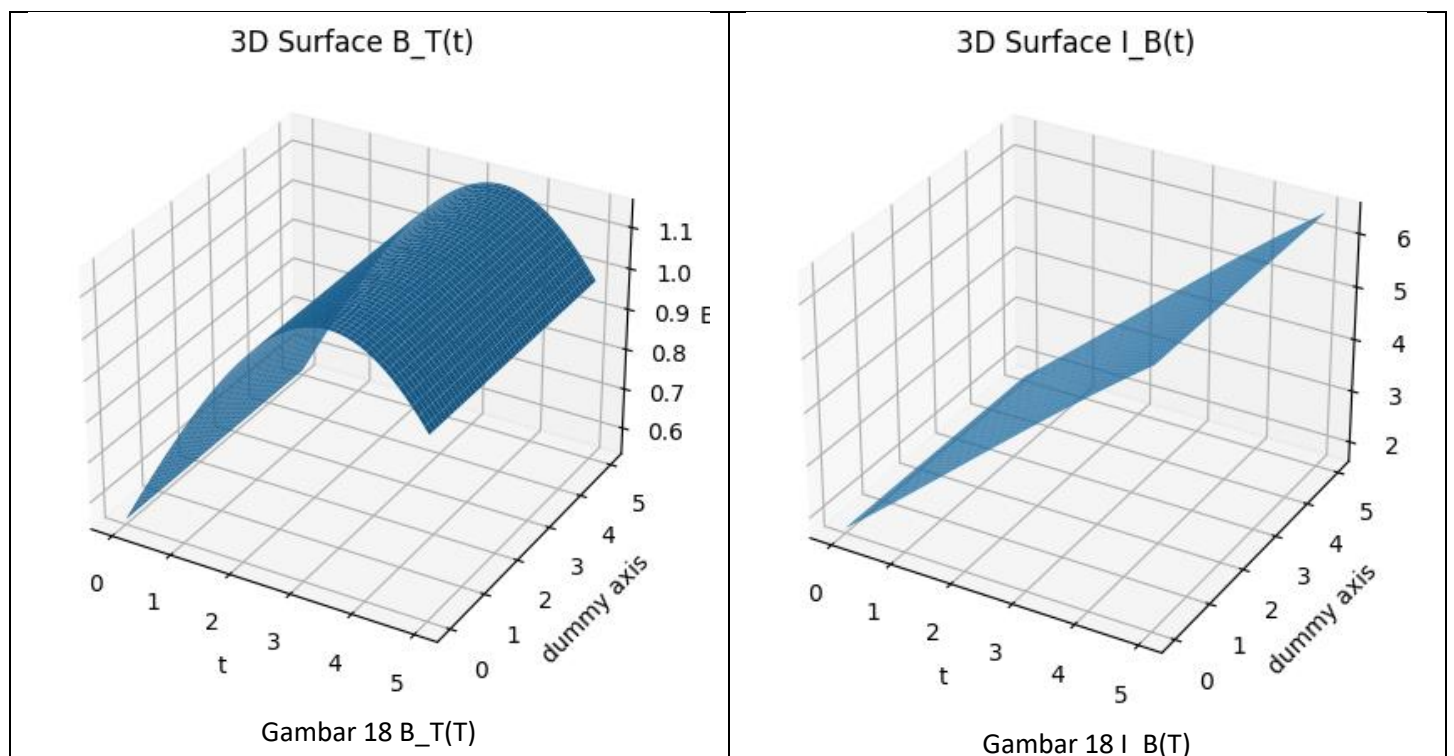
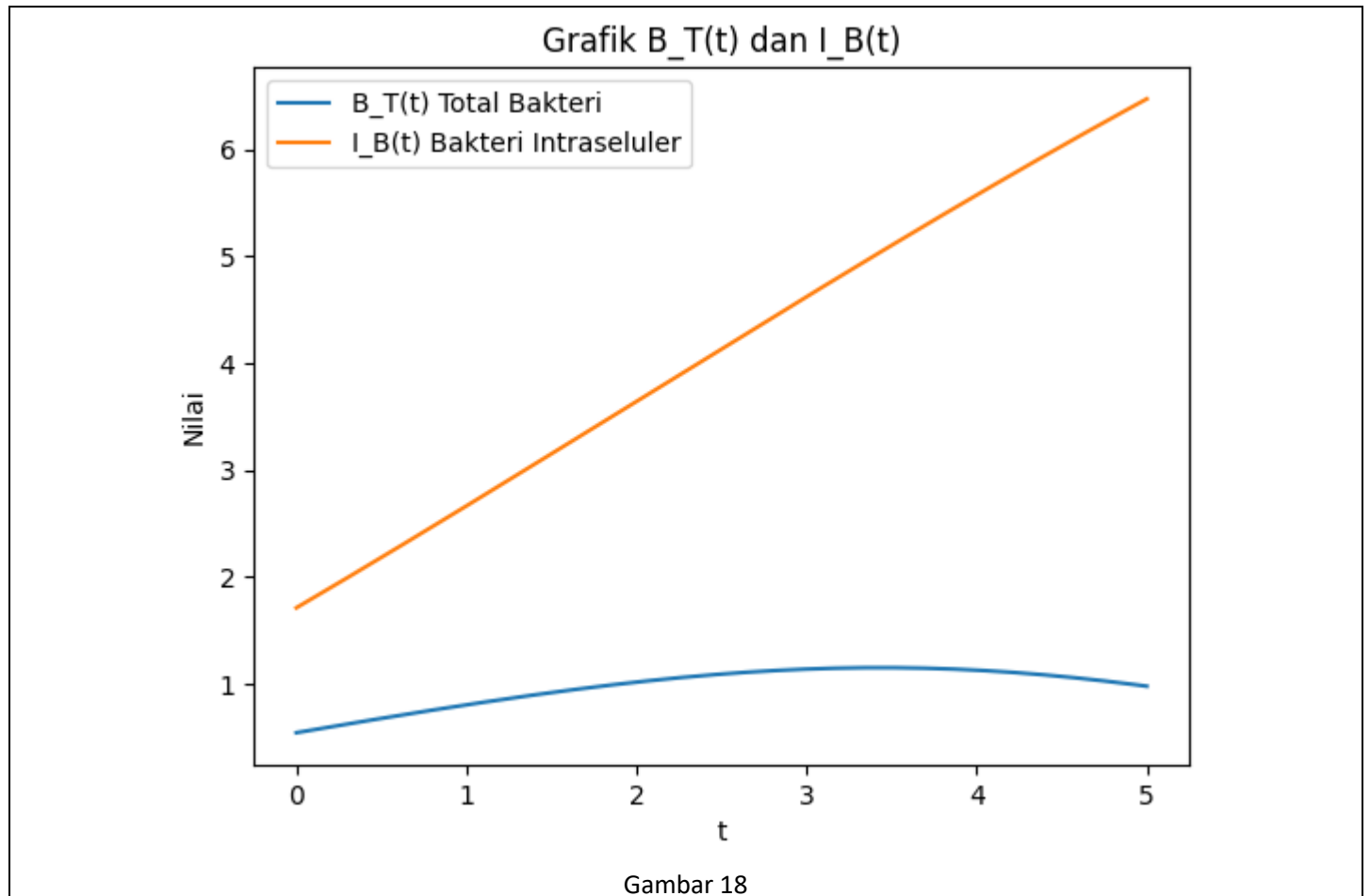
plt.show()

```

Persamaan 14:

$$\begin{aligned}
 B_T(t) &= \int_0^{R(t)} B(x, t) dx \\
 &= R(t) \int_0^1 B(\rho, t) d\rho, \\
 I_B(t) &= \int_0^{R(t)} V_R \sum_{w=0}^N w M_w(x, t) dx \\
 &= R(t) \int_0^1 V_R \sum_{w=0}^N w M_w(\rho, t) d\rho.
 \end{aligned} \tag{14}$$

Hasil persamaan



```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
```

```
# =====
```

```

# 1. DOMAIN
# =====
t = np.linspace(0, 5, 150)
y = np.linspace(0, 5, 80) # dummy axis untuk 3D

T, Y = np.meshgrid(t, y)

# =====
# 2. PARAMETER
# =====
VR = 0.5
N = 3

# =====
# 3. FUNGSI MODEL
# =====

# Radius granuloma
def R_func(t):
    return 1 + 0.5 * t

# Distribusi bakteri
def B_func(rho, t):
    return np.exp(-((rho - 0.5 - 0.1*t)**2) * 10)

# Distribusi makrofag terinfeksi
def Mw_func(rho, t, w):
    return np.exp(-((rho - 0.3 - 0.05*t)**2) * (5 + w))

# =====
# 4. GRID INTEGRASI
# =====
rho = np.linspace(0, 1, 100)

# =====
# 5. HITUNG BT dan IB
# =====
BT_surface = np.zeros_like(T)
IB_surface = np.zeros_like(T)

for i in range(T.shape[0]):
    for j in range(T.shape[1]):
        ti = T[i, j]

        # ===== BT =====
        B_vals = B_func(rho, ti)
        BT_surface[i, j] = R_func(ti) * np.trapz(B_vals, rho)

        # ===== IB =====
        Mw_sum = np.zeros_like(rho)
        for w in range(N+1):
            Mw_sum += w * Mw_func(rho, ti, w)

        IB_surface[i, j] = R_func(ti) * VR * np.trapz(Mw_sum, rho)

# =====
# 6. PLOT 3D BT
# =====

```



```
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

ax.plot_surface(T, Y, BT_surface)

ax.set_title("3D Surface B_T(t)")
ax.set_xlabel("t")
ax.set_ylabel("Dimensi tambahan")
ax.set_zlabel("B_T")

plt.show()

# =====
# 7. PLOT 3D IB
# =====
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

ax.plot_surface(T, Y, IB_surface)

ax.set_title("3D Surface I_B(t)")
ax.set_xlabel("t")
ax.set_ylabel("Dimensi tambahan")
ax.set_zlabel("I_B")

plt.show()
```